Name- Suyash Singh Tomar

Subject- Computer vision Lab

CUID-CU24250178

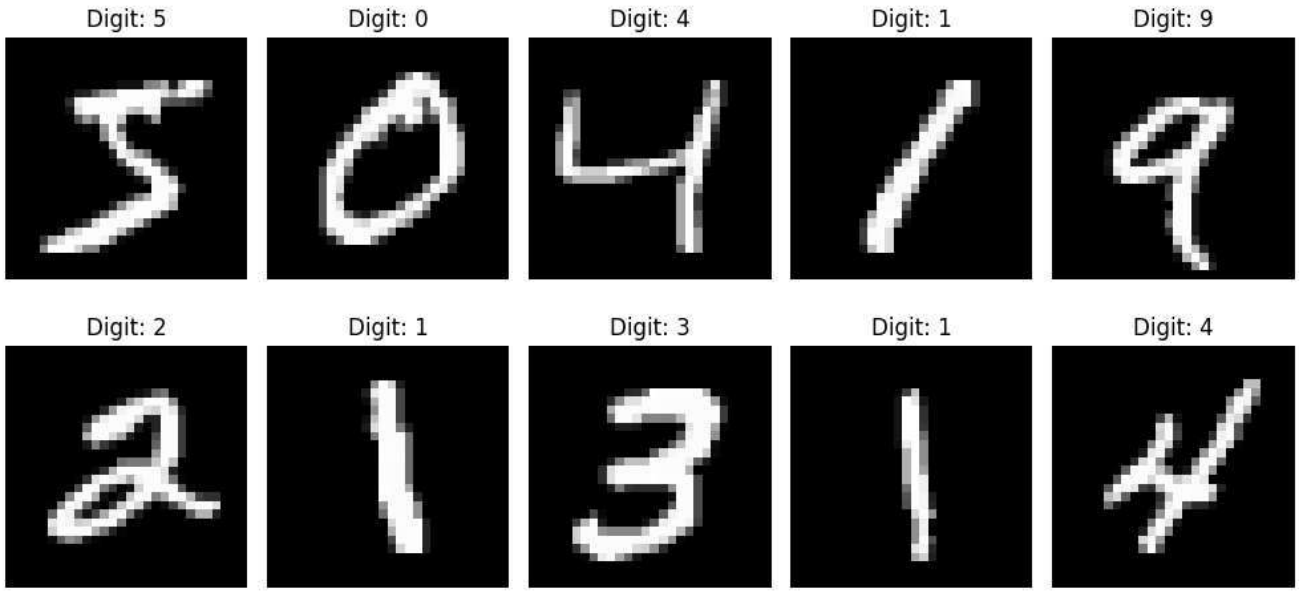
1. Select a standard dataset such as MNIST, Fashion-MNIST, EMNIST, or CelebA for the classification task.

```
import tensorflow as tf
import numpy as np
import matplotlib.pyplot as plt
from tensorflow.keras import datasets, layers, models
(x_train, y_train), (x_test, y_test) = datasets.mnist.load_data()
print("Training Images:", x_train.shape)
print("Training Labels:", y_train.shape)
print("Testing Images:", x_test.shape)
print("Testing Labels:", y_test.shape)
```

```
Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-datasets/mnist.npz
11490434/11490434 ━━━━━━━━━━━━━━━━━━━━ 0s 0us/step
Training Images: (60000, 28, 28)
Training Labels: (60000,)
Testing Images: (10000, 28, 28)
Testing Labels: (10000,)
```

2. Import the required libraries and load the selected dataset into the development environment.

```
plt.figure(figsize=(10, 5))
for i in range(10):
    plt.subplot(2, 5, i + 1)
    plt.imshow(x_train[i], cmap='gray')
    plt.title("Digit: " + str(y_train[i]))
    plt.axis('off')
plt.tight_layout()
plt.show()
```



3. Perform image preprocessing by resizing (if required), normalization, and splitting the dataset into training, validation, and testing sets.

```
x_train = x_train.astype('float32') / 255.0
x_test = x_test.astype('float32') / 255.0
x_train = x_train.reshape(-1, 28, 28, 1)
```

```
x_test = x_test.reshape(-1, 28, 28, 1)
print("Training shape:", x_train.shape)
print("Testing shape:", x_test.shape)
```

```
Training shape: (60000, 28, 28, 1)
Testing shape: (10000, 28, 28, 1)
```

4. Design a Convolutional Neural Network (CNN) consisting of convolutional layers, pooling layers, activation functions, fully connected layers, and an output layer.

```
model = models.Sequential([
    layers.Conv2D(32, (3, 3), activation='relu',
                  input_shape=(28, 28, 1)),
    layers.MaxPooling2D((2, 2)),
    layers.Conv2D(64, (3, 3), activation='relu'),
    layers.MaxPooling2D((2, 2)),
    layers.Flatten(),
    layers.Dense(64, activation='relu'),
    layers.Dense(10, activation='softmax')
])

model.summary()
```

```
/usr/local/lib/python3.13/dist-packages/keras/src/layers/convolutional/base_conv.py:113: UserWarning: Do not pass an `input_shap
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
Model: "sequential"
```

Layer (type)	Output Shape	Param #
conv2d (Conv2D)	(None, 26, 26, 32)	320
max_pooling2d (MaxPooling2D)	(None, 13, 13, 32)	0
conv2d_1 (Conv2D)	(None, 11, 11, 64)	18,496
max_pooling2d_1 (MaxPooling2D)	(None, 5, 5, 64)	0
flatten (Flatten)	(None, 1600)	0
dense (Dense)	(None, 64)	102,464
dense_1 (Dense)	(None, 10)	650

```
Total params: 121,930 (476.29 KB)
Trainable params: 121,930 (476.29 KB)
Non-trainable params: 0 (0.00 B)
```

5. Configure the model using an appropriate optimizer, loss function, learning rate, batch size, and number of training epochs.

```
model.compile(
    optimizer='adam',
    loss='sparse_categorical_crossentropy',
    metrics=['accuracy']
)``````````````````````````````````````````````````````````````````
print("CNN model compiled successfully!")
```

```
CNN model compiled successfully!
```

6. Train the CNN model using the training dataset and monitor the learning process through training and validation accuracy/loss curves.

```
history = model.fit(
    x_train,
    y_train,
    epochs=5,
    batch_size=64,
    validation_split=0.1
)
```

```
Epoch 1/5
844/844 ━━━━━━━━━━━━━━━━━━━━ 46s 53ms/step - accuracy: 0.9450 - loss: 0.1851 - val_accuracy: 0.9828 - val_loss: 0.0613
Epoch 2/5
844/844 ━━━━━━━━━━━━━━━━━━━━ 80s 51ms/step - accuracy: 0.9830 - loss: 0.0559 - val_accuracy: 0.9873 - val_loss: 0.0434
Epoch 3/5
844/844 ━━━━━━━━━━━━━━━━━━━━ 57s 67ms/step - accuracy: 0.9882 - loss: 0.0388 - val_accuracy: 0.9897 - val_loss: 0.0345
```

```
Epoch 4/5
844/844 ━━━━━━━━━━━━━━━━━━━━ 57s 67ms/step - accuracy: 0.9909 - loss: 0.0280 - val_accuracy: 0.9905 - val_loss: 0.0341
Epoch 5/5
844/844 ━━━━━━━━━━━━━━━━━━━━ 74s 58ms/step - accuracy: 0.9928 - loss: 0.0229 - val_accuracy: 0.9880 - val_loss: 0.0426
```

7. Evaluate the trained model on the testing dataset using performance metrics such as Accuracy, Precision, Recall, F1-Score, and Confusion Matrix.

```
test_loss, test_accuracy = model.evaluate(x_test, y_test)
print("Test Loss:", test_loss)
print("Test Accuracy:", test_accuracy)
from sklearn.metrics import classification_report, confusion_matrix
y_probability = model.predict(x_test)
y_pred = np.argmax(y_probability, axis=1)
print(classification_report(y_test, y_pred))
```

```
313/313 ━━━━━━━━━━━━━━━━━━━━ 3s 9ms/step - accuracy: 0.9899 - loss: 0.0321
Test Loss: 0.03206883370876312
Test Accuracy: 0.9898999929428101
313/313 ━━━━━━━━━━━━━━━━━━━━ 3s 8ms/step
              precision    recall  f1-score   support

           0       0.99      0.99      0.99       980
           1       1.00      0.99      0.99      1135
           2       0.98      1.00      0.99      1032
           3       0.97      1.00      0.98      1010
           4       1.00      0.99      0.99       982
           5       0.99      0.98      0.99       892
           6       0.99      0.99      0.99       958
           7       0.99      0.99      0.99      1028
           8       0.99      0.99      0.99       974
           9       0.99      0.98      0.99      1009

    accuracy                           0.99     10000
   macro avg       0.99      0.99      0.99     10000
weighted avg       0.99      0.99      0.99     10000
```

8. Perform predictions on unseen images and compare the predicted labels with the actual labels.

```
plt.figure(figsize=(12, 6))

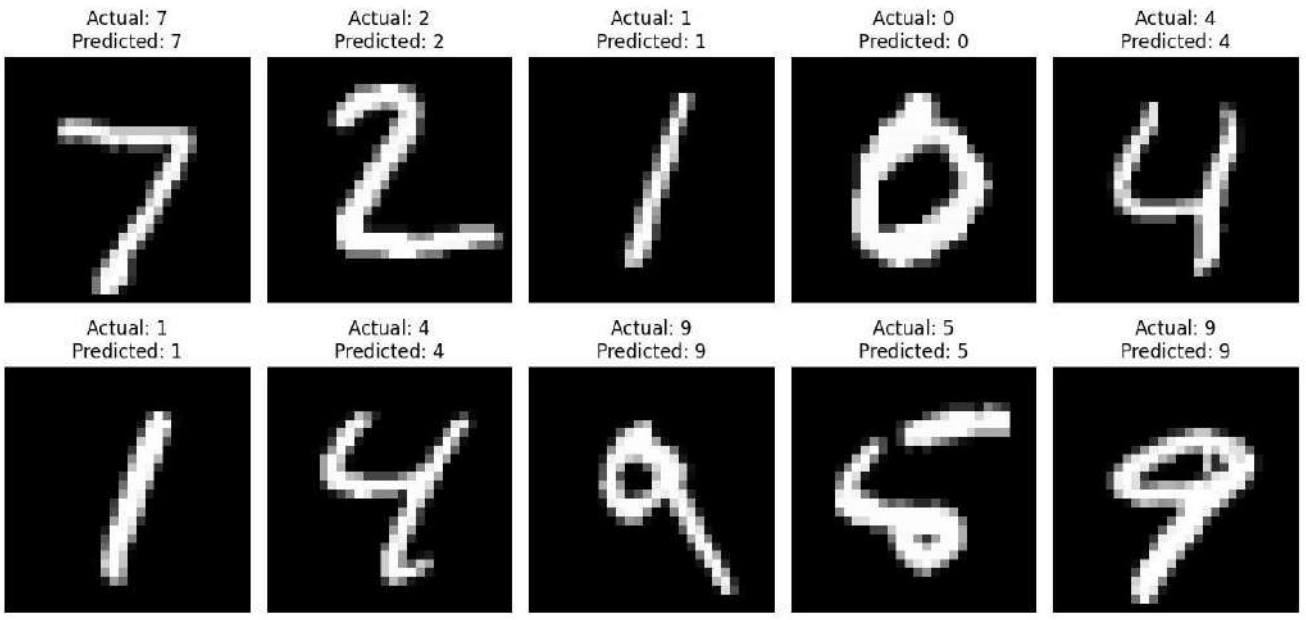
for i in range(10):
    plt.subplot(2, 5, i + 1)

    plt.imshow(x_test[i].reshape(28, 28), cmap='gray')

    plt.title(
        "Actual: " + str(y_test[i]) +
        "\nPredicted: " + str(y_pred[i])
    )

    plt.axis('off')

plt.tight_layout()
plt.show()
```



9. Analyze the model's performance, identify possible misclassifications, and discuss factors affecting classification accuracy.

```
wrong = np.where(y_pred != y_test)[0]

print("Total Misclassified Images:", len(wrong))
plt.figure(figsize=(12, 8))

for i in range(min(10, len(wrong))):
    index = wrong[i]

    plt.subplot(2, 5, i + 1)

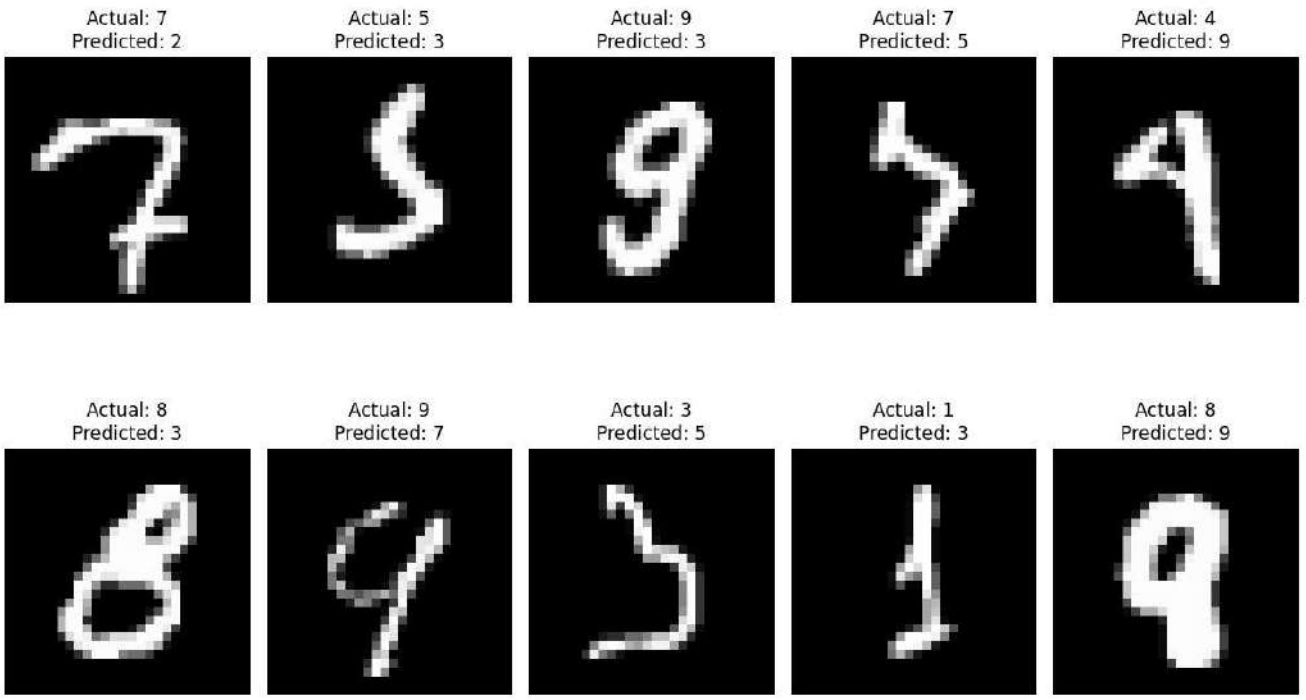
    plt.imshow(x_test[index].reshape(28, 28), cmap='gray')

    plt.title(
        "Actual: " + str(y_test[index]) +
        "\nPredicted: " + str(y_pred[index])
    )

    plt.axis('off')

plt.tight_layout()
plt.show()
```

```
Total Misclassified Images: 101
```



10. Summarize the results and discuss the practical applications of CNN-based image classification in modern computer vision systems.

```
plt.figure(figsize=(8, 5))

plt.plot(history.history['accuracy'], label='Training Accuracy')
plt.plot(history.history['val_accuracy'], label='Validation Accuracy')

plt.xlabel('Epoch')
plt.ylabel('Accuracy')
plt.title('Training and Validation Accuracy')

plt.legend()
plt.show()
plt.figure(figsize=(8, 5))

plt.plot(history.history['loss'], label='Training Loss')
plt.plot(history.history['val_loss'], label='Validation Loss')

plt.xlabel('Epoch')
plt.ylabel('Loss')
plt.title('Training and Validation Loss')

plt.legend()
plt.show()
from sklearn.metrics import confusion_matrix

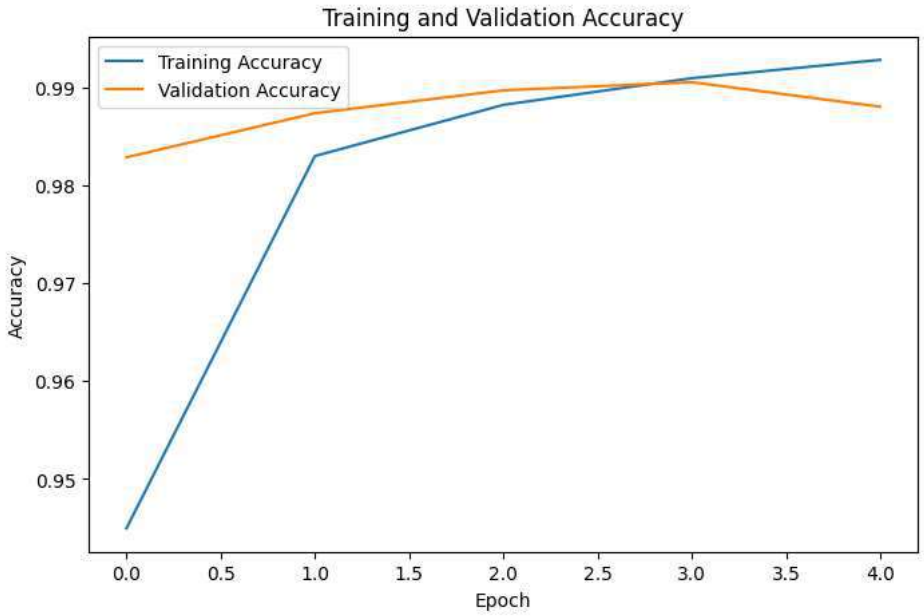
cm = confusion_matrix(y_test, y_pred)

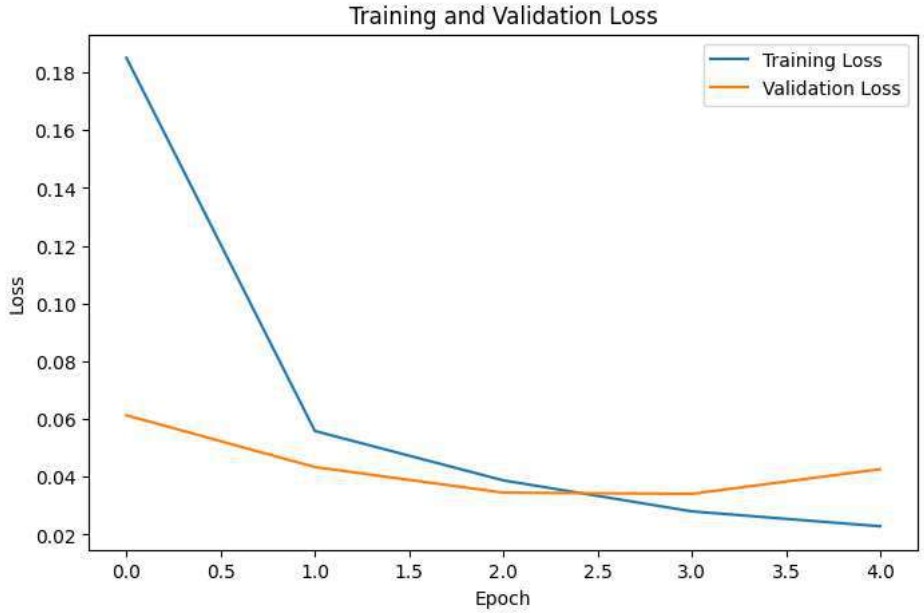
plt.figure(figsize=(10, 8))

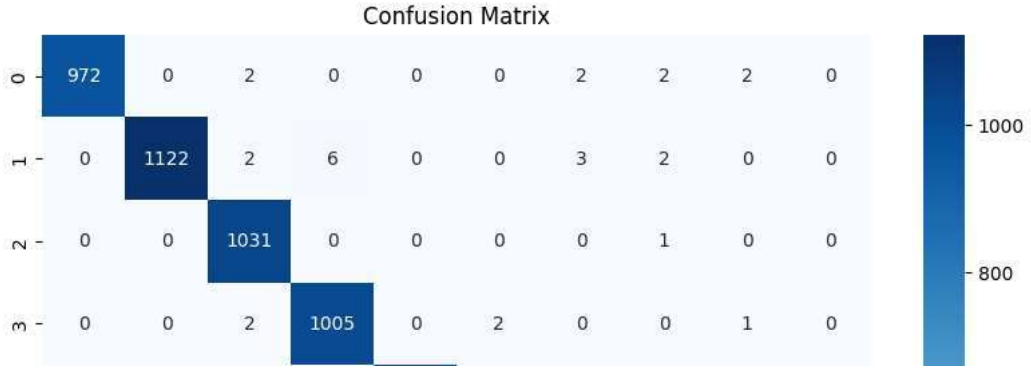
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues')

plt.xlabel('Predicted Label')
plt.ylabel('Actual Label')
plt.title('Confusion Matrix')

plt.show()
```


Training and Validation Accuracy
Training Accuracy
Validation Accuracy
Accuracy
Epoch


Training and Validation Loss
Training Loss
Validation Loss
Loss
Epoch


Confusion Matrix
0 972 0 2 0 0 0 2 2 2 0
1 0 1122 2 6 0 0 3 2 0 0
2 0 0 1031 0 0 0 0 1 0 0
3 0 0 2 1005 0 2 0 0 1 0
1000
800